

# Lecture13: Tree2

## Decision Tree

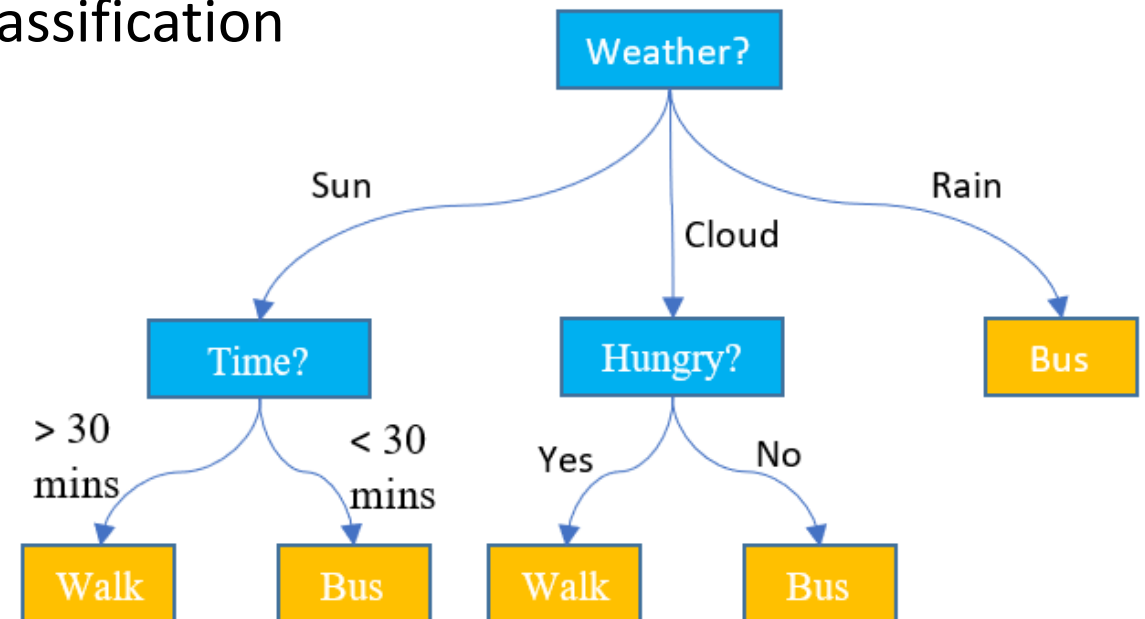
STA 4724

HanQin Cai

University of Central Florida

# Decision Tree

- Decision tree is another well-known classification approach.
  - Using a decision tree is straight forward
  - Start from the root node
  - Answer a series of questions
  - Ending at one of the leaf nodes
- The final decision is discrete, thus it is a classification



# The Nodes

- A good tree must have a solid root. The root node is the node that has no incoming edges, and two or more outgoing edges.
- An internal node has one incoming edge and two or more outgoing ones: Internal nodes represent test conditions at every given level.
- A leaf node has one incoming edge and no outgoing edge. Each leaf node is corresponding to class labels.

# Training

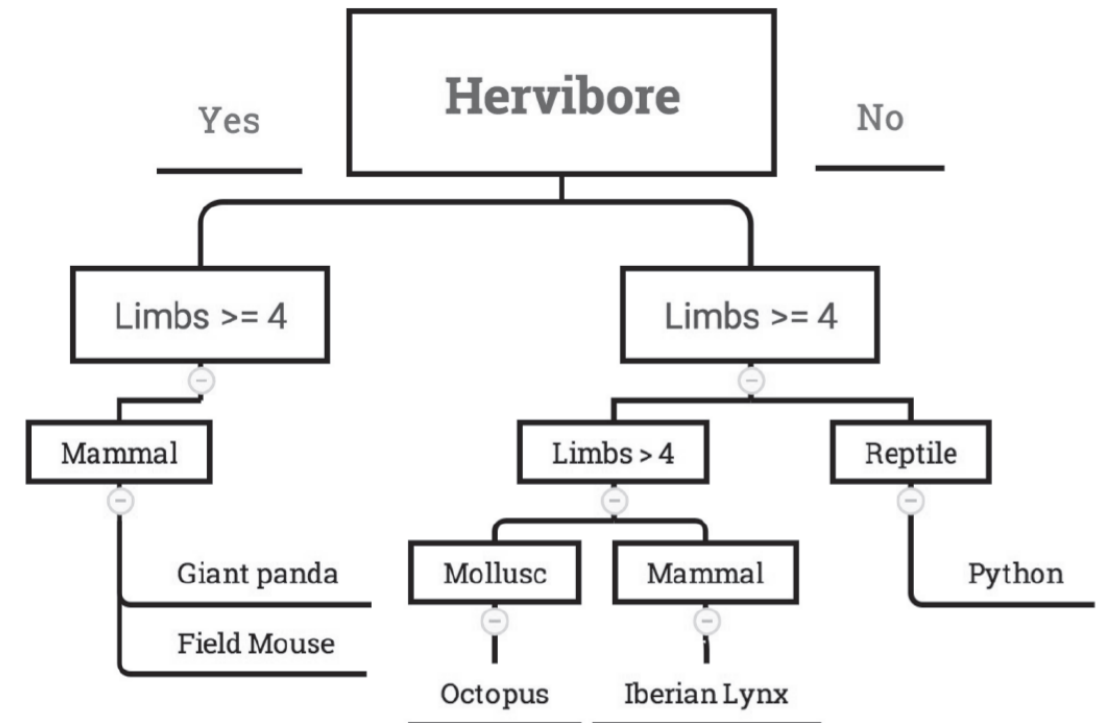
- While using the decision tree is easy, the question is how to train one.
- Let's the following dataset---classify the type of a given animal

| Animal       | Limbs | Herbivore | Class   |
|--------------|-------|-----------|---------|
| Python       | 0     | No        | Reptile |
| Iberian Lynx | 4     | No        | Mammal  |
| Giant Panda  | 4     | Yes       | Mammal  |
| Field Mouse  | 4     | Yes       | Mammal  |
| Octopus      | 8     | No        | Mollusc |

# Training

- It is possible to evaluate all possible combinations of the test conditions in such a small dataset
- For example, we can start by testing whether or not the animal is herbivore.
- We can continue by asking how many limbs the animal has, and so on

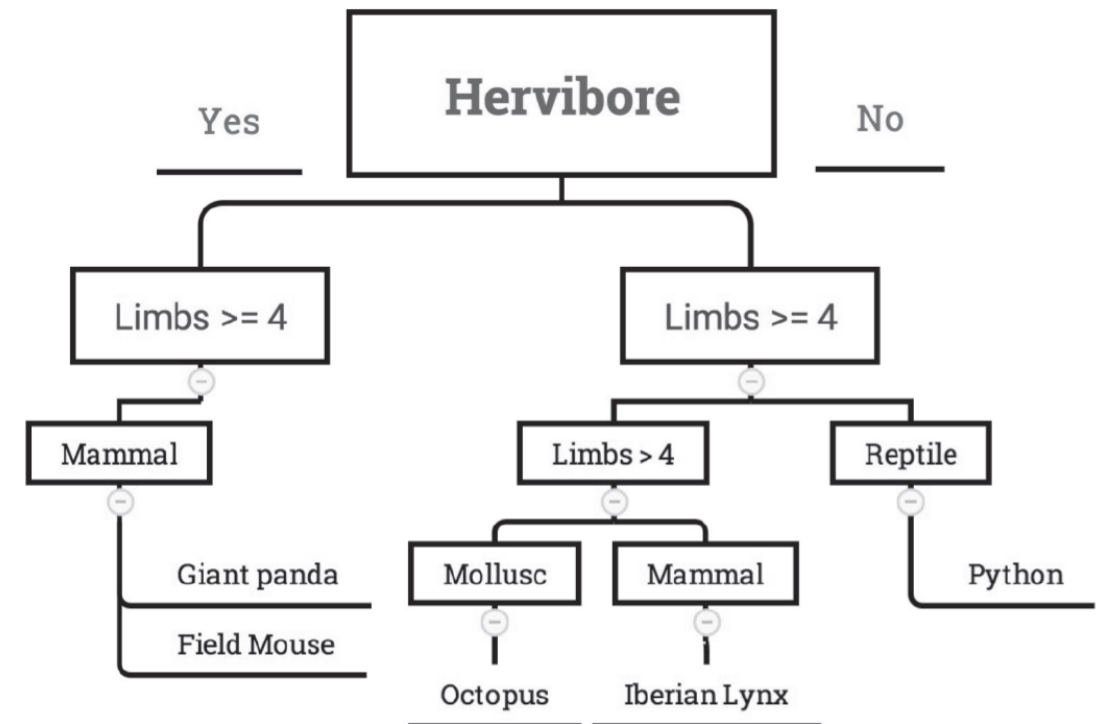
| Animal       | Limbs | Herbivore | Class   |
|--------------|-------|-----------|---------|
| Python       | 0     | No        | Reptile |
| Iberian Lynx | 4     | No        | Mammal  |
| Giant Panda  | 4     | Yes       | Mammal  |
| Field Mouse  | 4     | Yes       | Mammal  |
| Octopus      | 8     | No        | Mollusc |



# Training

- Obviously, not all the tree are as good as each other.
- However, constructing all possible decision trees is not practical
- We want a greedy algorithm that lets us find a local solution quickly

| Animal       | Limbs | Herbivore | Class   |
|--------------|-------|-----------|---------|
| Python       | 0     | No        | Reptile |
| Iberian Lynx | 4     | No        | Mammal  |
| Giant Panda  | 4     | Yes       | Mammal  |
| Field Mouse  | 4     | Yes       | Mammal  |
| Octopus      | 8     | No        | Mollusc |



# Hunt's algorithm

- Construct a decision tree in a top-down fashion
  - Recursively splitting the dataset into smaller and smaller data subsets
- To minimize the cost of the classification task: Balancing the classification accuracy and the conciseness of the tree structure
- Selecting a test condition that leads to smaller subsets of data is a good idea. However, the smaller subsets should actually lead to leaf nodes that classify data instances accurately
- Introduce a measurement: *purity*
  - A subset is said to be 100% if all its data points belong to the same class
  - It means that it is not necessary to split the data any further

# Hunt's algorithm

- Animals described in that table as having 4 limbs are all mammals
  - Splitting the data by the number of limbs is a good start point

| Animal       | Limbs | Herbivore | Class   |
|--------------|-------|-----------|---------|
| Python       | 0     | No        | Reptile |
| Iberian Lynx | 4     | No        | Mammal  |
| Giant Panda  | 4     | Yes       | Mammal  |
| Field Mouse  | 4     | Yes       | Mammal  |
| Octopus      | 8     | No        | Mollusc |

# Hunt's algorithm

For a binary classification problem with classes  $A$  and  $B$ , and a dataset with  $N_n$  data instances at node  $n$

1. If node  $n$  is pure, i.e., all its  $N_n$  data instances belong to class  $A$  (or  $B$ ), we are done
  - $n$  is a leaf node
2. If node  $n$  is not pure, we need to split the dataset further. We need to create a test condition that enables a partition in the dataset
  - $n$  is an internal node
3. We run the test condition and assign each of the  $N_n$  instances to one of the two child nodes created from node  $n$
4. These steps are applied recursively to each and every child node until all leaf nodes are pure

# Entropy

- Step 2 of hunt's algorithm states that we need to create a test condition. We prefer a condition that gives us the best split in terms of purity for each of the child nodes
- Entropy is a good measurement for the quality of a test condition:

$$E(n) = - \sum_{i=1}^c p(i|n) \log_2 p(i|n)$$

- where  $c$  is the total number of involved classes,  $p(i|n)$  denotes the fraction of instances that belong to class  $i$  at a given node  $n$ 
  - At high level, entropy describes how chaos/impurity it is in a node
  - The entropy of a pure node is 0
  - For binary classifier, the worst entropy in a node is 1
- Exercise: How do you get entropy = 1 in the example?

# Gini Impurity

- The Gini impurity is also a popular measurement
- Gini impurity:

$$G(n) = 1 - \sum_{i=1}^c p^2(i|n)$$

- where  $c$  is the total number of classes,  $p(i|n)$  denotes the fraction of instances that belong to class  $i$  at a given node  $n$ 
  - At high level, Gini impurity describes how impure it is in a node
  - The Gini impurity of a pure node is 0
  - The worst Gini impurity in a node is 0.5, regardless of the number of classes involved (warning: the textbook is wrong)

# Classification Error

- The classification error is also another popular impurity measurement

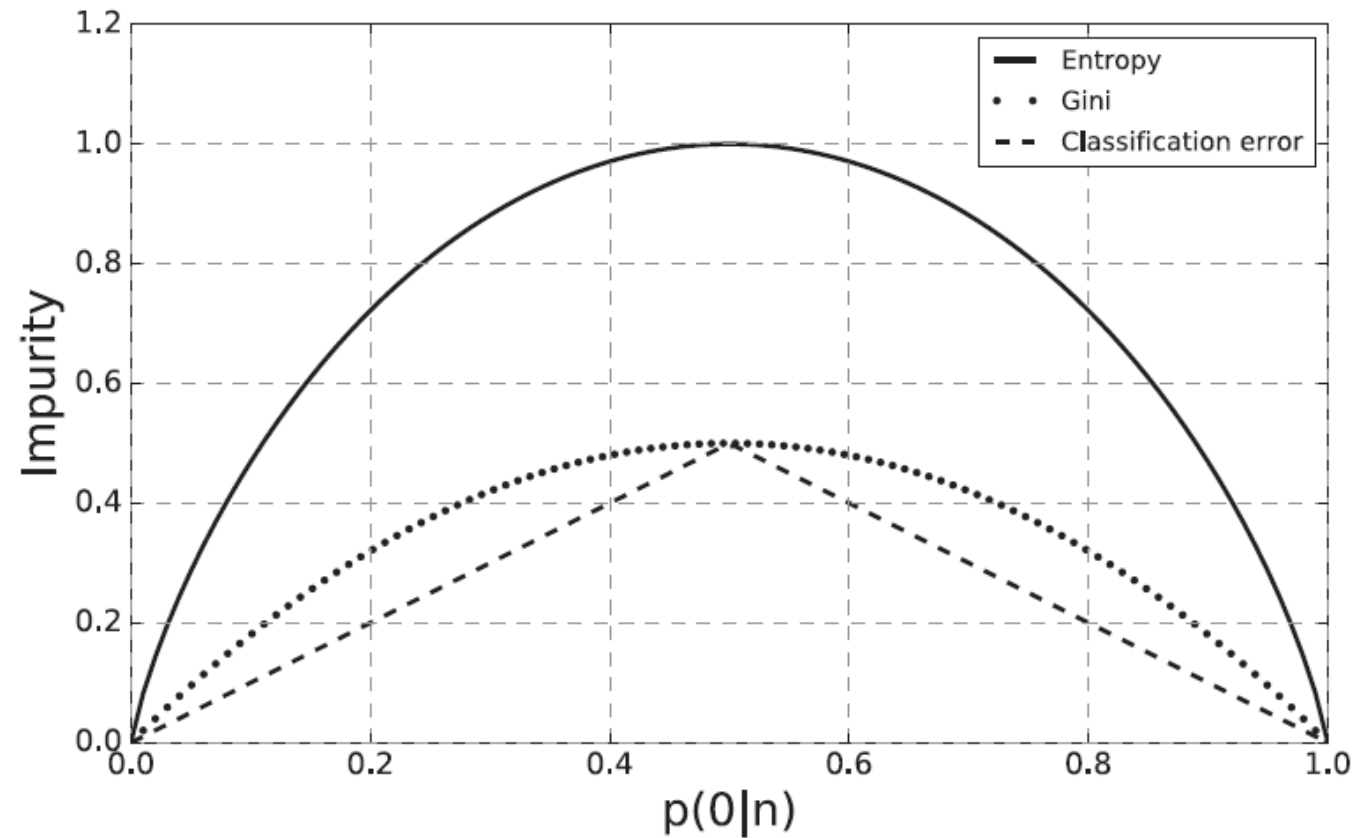
- Classification error:

$$E(n) = 1 - \max_i p(i|n)$$

- where  $p(i|n)$  denotes the fraction of instances that belong to class  $i$  at a given node  $n$ 
  - The classification error of a pure node is 0
  - The worst classification error in a node is  $1 - \frac{1}{c}$  with total  $c$  involved classes (warning: and the textbook is wrong again)

# Compare the 3 Impurity Measurement

- Comparison for a binary classifier



# Gain

- With any of the impurity functions  $I(n)$  mentioned above, we are now in a position to compare the purity of the parent and children nodes with the Gain:

$$\Delta = I(\text{parent}) - \sum_{\text{children}} \frac{N_{j^{th} \text{ child}}}{N_{\text{parent}}} I(j^{th} \text{ child})$$

- When we choose the entropy to be our impurity measurement, the Gain is actually called the ***Information Gain***

# Overfitting

- Decision tree algorithms are not exempt from overfitting
- In the worst case scenario, we can end up with a number of leaves equal to the number of data instances.
- We can restrict our tree to binary splits exclusively (CART algorithm) or by imposing a penalty on the number of outcomes ( $C_{4.5}$  algorithm).

- A popular regularization is the Gain Ratio

$$GR = \frac{\Delta_{info}}{-\sum_{children} \frac{N_{j^{th} \ child}}{N_{parent}} \log_2 \left( \frac{N_{j^{th} \ child}}{N_{parent}} \right)}$$

- The Gain Ratio reduces the bias towards multi-valued features as it considers the number and size of nodes when choosing a feature for splitting our data.

# Time to Play

- Let's see how to use Hierarchical Clustering in **python**!

